

RideWheelShare — Production N+1 & Performance Analysis

Read-only code investigation · App root: /var/www/app.ridewheelshare.com · Laravel 9.52 / PHP 8.2 · Prepared: 2026-07-28 · For: Development Team

Why this report exists: The production app server (t3.xlarge) hits 100% CPU every business day. On Jul 27: 399,559 requests, **1,016 × HTTP 503** and **273 × HTTP 500** served to users. ALB-measured cost ranking shows `GET api/driverbooking/{id}` at **9–11 seconds per call** (~28 CPU-hours that day alone). Database is idle (RDS at ~5%); all time is burned in PHP executing per-row query loops (N+1). ProxySQL digest stats over 26 days: **143M** single-key `user_meta` lookups, **97M** role display-name joins, **43M** `ride_status` lookups.

1. Route → Controller Map

Correction: route name `driverbooking.edit` maps to `AuthController@driverbooking`, not `@edit` (no `edit` method exists on `AuthController`).

Endpoint	Route file	Handler
<code>GET api/driverbooking/{id}</code>	<code>routes/api.php:46</code>	<code>AuthController::driverbooking</code> (L119–260)
<code>POST api/dashboardappnew</code>	<code>routes/api.php:223</code>	<code>ApiController::dashboardappnew</code> (L14010–14178)
<code>POST api/pendingapplistingapidata</code>	<code>routes/api.php:259</code>	<code>AuthController::pendingapplistingapidata</code> (L3267–3424)
<code>POST api/appdashboardcalenderlistingday</code>	<code>routes/api.php:251</code>	<code>AuthController::appdashboardcalenderlistingday</code> (L1252–2061)
<code>POST api/drivercolorapi</code>	<code>routes/api.php:292</code>	<code>ApiController::drivercolorapi</code> (L15901–15983)
<code>GET /check-new-bookings</code>	<code>routes/web.php:436</code>	<code>BookingController::checkNewBookings</code> (L1746)
<code>GET /nemt-bookings</code>	<code>routes/web.php:437</code>	<code>BookingController::checkNewBookingsNemt</code> (L1751)
<code>GET /fareupdatenotify</code>	<code>routes/web.php:515–523</code>	Inline closure (<code>FacilityPricingTemp::exists</code>)
<code>POST api/bookingslistapp</code> (related)	—	<code>AuthController::bookingsapplist</code> (L441+) — <code>Safetybookingcheck::where()->first()</code> inside booking loops at L545/713/876; matches the 6.6M <code>safetybookingcheck</code> query stats

2. Exact N+1 Locations

2.1 GET api/driverbooking/{id} — PRIMARY CPU OFFENDER (9–11 s/call)

Upfront global loads (AuthController.php L190–193): `getAllUsersByRoleReporting()` is called **three times** (Facility, Customer, Driver). Helper at `app/helpers.php:53–63`:

```
function getAllUsersByRoleReporting($roleName) {
    $users = App\Models\User::whereHas(
        'roles', function($q) use ($roleName) {
            $q->where('name', $roleName);
        }
    )->with('usersMetas')
    ->get();
    return $users;
}
```

That loads **all users of that role system-wide + all their user_meta rows on every request** — and the response also returns the full `$customers` collection (L254).

Per-booking loop (AuthController.php L195–249):

Query / work	Lines	Per booking
<code>UserMeta::where(user_id, hospital_patient_room_number)->value()</code>	207–209	1
<code>UserMeta::where(..., wheelshare_facility_patient_room_number)</code>	211–213	1
<code>UserMeta::where(..., passenger_last_name)</code>	215–217	1
<code>UserMeta::where(user_id)->get()</code> (full meta dump)	225	1
Nested scan of all customers / facilities / drivers	203–242	CPU O(C+F+D)
<code>RideStatus::where('booking_id', \$booking->id)->first()</code>	244	1

If `limit / offset` are omitted, the else branch (L161–187) loads **every non-cancelled booking for the driver**, multiplying the above by N. Estimated: ~1 booking SELECT + 3 role/whereHas loads + eager meta + ~5N queries. For N≈500–2000 → thousands of queries → matches the observed 9–11 s.

2.2 POST api/appdashboardcalenderlistingday

Main booking fetch already uses JOINS on `user_meta` (good). The N+1 is the **role display-name lookup inside every booking iteration** (AuthController.php L1495–1509):

```
$rolebookingcreatename = DB::table('role_user')
->join('roles', ...) ->join('users', ...)
->where('role_user.user_id', $booking->created_by)
->selectRaw("CONCAT(...) as display_name")
->value('display_name');
```

Repeated at L1509, L1749, L2012 (TM / Driver / Facility branches) and in the week/month sibling methods (L2317, 2573, 2888, 3152, ...). **This is the smoking gun for ~97M role display-name joins**. Also: `whereHas('roles')` for driver/facility lists (L1285+, L1553+) and nested `foreach ($facilities)` per booking (PHP-side CPU).

2.3 POST api/drivercolorapi

Classic N+1 inside `->map()` (ApiController.php L15928–15931, duplicated ~L15955, ~L15975 for the three role branches):

```
'passenger_last_name' => DB::table('user_meta')
->where('user_id', $driver->id)
->where('meta_key', 'passenger_last_name')
->value('meta_value'),
```

~(1 + D) meta lookups per call × ~39,000 calls/day.

2.4 POST api/dashboardappnew

Facility/TM paths: fixed set of COUNT/SUM/JOIN queries (~5–7). Latency (~500 ms × 23k/day) driven by heavy aggregates on bookings + role_user joins, not per-row loops. Superadmin/else path (L14126+) loads `getAllTransactions()/getAllExpenses()` and sums in PHP — heavy but rarely hit by mobile polling.

2.5 POST api/pendingapplistingapidata

Already batched via JOINS — no per-row loops. **Cache bug:** L3415–3419 performs `Redis::set + expire(60)` but **there is no `Redis::get` anywhere** under app/ or routes/. Every request still hits the DB; the Redis write adds load without ever serving a read.

2.6 Polling web endpoints

- `checkNewBookings`: `Booking::getBookingsWithoutDrivers()` loads the **full row collection** then `->count()` in PHP (Booking.php L82–103; controller L1746). Also lazy-loads `Auth::user()->roles`.
- `checkNewBookingsNemt`: one COUNT join — OK; still lazy-loads roles.
- `fareupdatenotify`: single `exists()` — fine.

3. Model Relationship Gaps (block clean eager loading)

Model	Issue
UserMeta	No <code>belongsTo(User)</code>
RideStatus	No <code>belongsTo(Booking)</code>
Safetybookingcheck	No <code>belongsTo(Booking)</code>
Booking	Has <code>customer()</code> , <code>rideStatusbookinglistapp()</code> , <code>driverbookinglistapp()</code> — missing <code>facility()</code> , <code>canonical driver()</code> , <code>rideStatus()</code> , <code>safetyCheck()</code> , <code>creator()</code>
User	<code>usersMetas()/meta()</code> OK; <code>getMetaValueByKey()</code> (L198–200) still runs a fresh query each call

Note: `driverbooking` never uses the existing `rideStatusbookinglistapp()` relation — it queries `RideStatus` imperatively.

4. Existing Caching Status

- Hot paths: **no** `Cache::remember / ->remember()` anywhere.
- Only Redis usage is the write-only set in `pendingapplistingapidata` (never read back).
- **Infrastructure ready:** `CACHE_DRIVER=redis`, `SESSION_DRIVER=redis`, `QUEUE_CONNECTION=redis`, `REDIS_HOST=127.0.0.1 (phpredis)`, DB via `ProxySQL :6033`.

5. Scheduled Tasks

All `->daily()`: `transfer:wallet`, `subscription:Renewable`, `driver:rating`, `booking:expired`. Not the cause of business-hour saturation; API polling is the main story.

6. Recommended Fixes (sketches — not applied)

6.1 driverbooking — highest ROI

```
$bookings = Booking::query()
->join('users as drivers', 'drivers.id', '=', 'bookings.driver_id')
->where('bookings.driver_id', $id)
->whereNotIn('bookings.booking_status', [2, 10])
->orderByDesc('bookings.pickup_datetime')
->when($limit !== null, fn ($q) => $q->limit($limit)->offset($offset ?? 0))
->select('bookings.*', 'drivers.name as driverNamez' /*, transport CASE */)
->with(['customer.usersMetas', 'rideStatusbookinglistapp'])
->get();

// Or batch meta without relations:
$customerIds = $bookings->pluck('customer_id')->filter()->unique();
$meta = UserMeta::whereIn('user_id', $customerIds)
->whereIn('meta_key', ['hospital_patient_room_number',
    'wheelshare_facility_patient_room_number', 'passenger_last_name'])
->get()->groupBy('user_id');
$rideStatuses = RideStatus::whereIn('booking_id', $bookings->pluck('id'))
->get()->keyBy('booking_id');
```

Also: stop returning the entire `$customers` collection in the JSON unless the client truly needs it.

6.2 appdashboardcalenderlistingday (and week/month siblings)

```
$creatorIds = $bookings->pluck('created_by')->unique();
$displayNames = DB::table('role_user')
->join('roles', ...) ->join('users', ...)
->whereIn('role_user.user_id', $creatorIds)
->selectRaw("role_user.user_id, CONCAT(...) as display_name")
->pluck('display_name', 'user_id');
// then $displayNames[$booking->created_by] inside the loop
```

6.3 drivercolorapi

```
$drivers = User::select('id', 'name', 'drivercolor')
->whereHas('roles', ...)
->where('facility_id', $facilityId)
->with(['usersMetas' => fn ($q) => $q->where('meta_key', 'passenger_last_name')])
->get(); // or leftJoin user_meta once
```

6.4 dashboardappnew

Cache the JSON payload 15–30 s keyed by role + facility_id + Y-m. Optional indexes on (facility_id, booking_status, payment_status, ride_type, pickup_datetime).

6.5 pendingapplistingapidata

Replace the write-only Redis set with `Cache::remember("PendingbookingList_{$facilityId}", 15, fn () => ...)` that is actually read.

6.6 check-new-bookings

```
return response()->json([
    'count' => Booking::query()->where(/* pending conditions */) ->count()
]);
// plus eager-load Auth::user()->load('roles')
```

7. Response-Cache Suitability (15–30 s TTL)

Endpoint	Cache?	Notes
dashboardappnew	Yes — strong candidate	Counts/sums; key: facility_id + role + month
drivercolorapi	Yes	Driver id/name/color/last name; invalidate on driver profile/color edits
pendingapplistingsapidata	Yes, short (10–15 s)	Changes when drivers assigned; intended Redis cache is currently write-only
appdashboardcalenderlistingday	Yes, short (10–20 s)	Calendar snapshot; key: facility + role + window
driverbooking/{id}	Cautious 5–15 s	Live ride list; key: driver_id + limit + offset; do not cache full customer_metadata dumps with payment/PII longer than necessary
check-new-bookings / nemt-bookings	Yes, 5–10 s	Badge counts only
fareupdatenotify	Optional 10–30 s	Boolean hasRequest

Must NOT cache (or must exclude from cached payloads): auth tokens, OTPs / 2FA codes, Stripe/payment method bank details, wallet amounts used for charges, full customer_metadata blobs containing card / connected_account keys, anything from payment success flows. Prefer projecting only the fields the app UI needs.

8. Estimated Query-Count Reduction

Assumptions: N = bookings returned, D = drivers in facility, C = customers matched.

Fix	Before (order of magnitude)	After	Reduction
driverbooking eager/batch	~1 + 3 role loads + 5N (+ giant meta eager of all customers)	~3–6	~95–99% of per-request queries; largest CPU win
Calendar display_name batch/join	1 + N role joins (×3 branches, × week/month siblings)	1	N → 1 per request; explains most of the 97M role joins
drivercolorapi join/eager	1 + D	1–2	~D × 39k/day meta lookups eliminated
dashboardappnew 15–30 s cache	5–7 × 23k/day	5–7 × misses only	~50–90% of DB hits skipped
pending... Cache::remember (fix write-only)	1 heavy join × 23k/day	join × misses only	~50–90% DB hits
check-new-bookings COUNT	full row load + PHP count	1 COUNT	Large row-transfer + PHP CPU cut

9. Priority Order for Implementation

1. **Rewrite AuthController::driverbooking** (global user dumps + 4× UserMeta + RideStatus per row) — sole explanation for the 9–11 s response time.
2. **Batch/join creator display_name** in calendar day/week/month methods.

3. **Fix drivercolorapi meta N+1.**
4. **Replace write-only Redis with Cache::remember** on dashboard / pending / color endpoints.
5. **Fix getBookingsWithoutDrivers to a COUNT query.**
6. **Add missing Eloquent relations** (UserMeta → User, RideStatus → Booking, Safetybookingcheck → Booking, Booking → facility/driver/rideStatus/creator) so future code can eager-load instead of querying imperatively.

Supporting production evidence (measured, Jul 27): total 399,559 requests; driverbooking consumed ~99,900 backend-seconds (≈28 CPU-hours) across ~14,200 calls; dashboard trio ~33,500 seconds combined; 1,016×503 + 273×500 errors served. EC2 peaked at 100% repeatedly 15:00–22:00 UTC while RDS stayed under 8%. Traffic is growing ~14%/week as facilities onboard. Investigation was strictly read-only; no files or services were modified.